

本物のプログラマーは `unsafe` と 型システム を使う

拡張可能レコードを作って学ぶ黒魔術入門

自己紹介

いし い ひろ み

- 石井 大海 / konn (Twitter: @mr_konn)
 - 博士（理学）。専門は数理論理学（特に集合論）と計算機代数（Gröbner基底とか）
 - 最近はプログラミング言語理論の周辺にいる気がします
 - 宣伝：EGRAPHS '26 で **e-graph の数理モデラーへの応用**について発表したり、PLDI '26 で 松下祐介氏と **Haskell 内で Rust 流の借用システムを実現する共同研究**を発表したりしました
- 現職：株式会社 Jij ソフトウェア開発チーム（2024/11～）
 - **Rust** で**数理最適化の埋込ドメイン特化言語 (EDSL)** を開発しています
- 前職では **Haskell** で数値計算向け **EDSL** を開発

本題

本物のプログラマーは

unsafe

と

型システム

を使う

本物のプログラマーは

unsafe

「キケン」な機能への
バックドア

と

型システム

を使う

本物のプログラマーは

unsafe

「キケン」な機能への
バックドア

と

型システム

バックドアを
「安全」に包むもの

を使う

本物のプログラマーは

講演後の
皆さんのこと

unsafe

「キケン」な機能への
バックドア

と

型システム

バックドアを
「安全」に包むもの

を使う

unsafe とは？

- 本講演でのunsafeの定義＝型システムの**検査を意図的に逃がれ**、それ単体では**安全性を保障できない**ようなプログラムを記述するための**エスケープハッチ**
 - Rust の `unsafe {}` ブロック / `unsafe fn` など
 - 所有権・借用検査をスキップして生のポインタにも触れるように
 - Haskell の `unsafePerformIO` や `unsafeCoerce` など
 - **IO 処理**をその場で（**純粋な計算として**）実行したり、ある型の値を**別の型にムリヤリ変換**したり
- 本講演では**もっぱら Haskell について**扱いますが、Rust でも思想は同じ

unsafe にまつわる迷信

😓 unsafe は「キケン」なので良くない

😄 いくら「強力な型システム」があっても **unsafe があっちゃ意味ない**じゃん

😡 **unsafe は罪！** unsafe 追放！

👻 unsafe がエルフの村を焼いたらしい

😞 unsafe はうちの裏には要らない

本当に？

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！（`unsafeCoerce` の **unsafe たるゆえん**）
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全だが……
 - ① **仕様に対する誤り**: `sum [] = 0; sum (x:xs) = x * sum xs`
 - ② **内部でキケンな操作**をしている: `sum xs = head xs + sum (tail xs)`

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！（`unsafeCoerce` の **unsafe たるゆえん**）
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全 
 - ① **仕様に対する誤り**: `sum [] = 0; sum (x:xs) = x * sum xs`
 - ② **内部でキケンな操作**をしている: `sum xs = head xs + sum (tail xs)`

Unsafe な実装が「キケン」なのはいつか

「キケン」にも色々ある

- そもそも実装しようとしている **API 自体がキケン**なもの
 - 例： `unsafeCoerce :: a → b` があると任意の型の値が手に入る！（`unsafeCoerce` の **unsafe たるゆえん**）
- API 自体はアンゼンだが**実装にミスがある**場合
 - 例： `sum :: [Int] → Int` はそれ型それ自体は安全 掛けちゃだめ！
 - ① **仕様に対する誤り**： `sum [] = 0; sum (x:xs) = x * sum xs`
 - ② **内部でキケンな操作**をしている： `sum xs = head xs + sum (tail xs)` 空リストで実行時エラー！

TODO: そもそも「キケン」とはどういう意味か？

「キケン」にも色々ある

- TODO: correctness vs soundness の話をする
- unsound の例として unsafeCoerce と head を挙げる

「不健全」の射程

unsafeCoerce も head も「キケン」！？

- ・ 型システムの健全性（型安全性とも）
 - ・ 標語：「型検査を通過したプログラムは**ヘンな振る舞い**をしない」
 - ・ ヘンな振る舞いとは？
 - ・ 最低限の要求：評価の前後で型が保たれること（保存）や、無限ループになってもいいから必ず評価が進むこと（進行）
- ・ 言語の「思想」や使い手の興味によって更に条件が増える
 - ・ Rust の所有権や Linear Haskell：「同じリソースに競合が起きない」まで保証したい！
 - ・ unsafeCoerce は明らかに不健全。なぜなら任意の値を任意の型に変換できると最悪SEGVるから
 - ・ head は尺度によって不健全。なぜなら head [] は実行時エラーになるから
- ・ ふわっと「型システムが当然保証してくれると思われる性質が破れてる」ことを不健全性と呼ぶ、と思えばよい

「unsafe は「キケン」なので良くない」

- head や tail、配列へのランダムアクセスだって「**キケン**」
- でも「そこは保証しない」というデザインチョイスになっている
 - 極力避けたいのでHaskell では非空性が保証された NonEmpty というリストの亜種を可能な限り使うことが最近では一般化しつつある
-

Case Study: 拡張可能レコード

Case Study: 拡張可能レコード

- レコード：フィールドと値の束
 - Haskell（や Rust の struct）ではフィールドと値の組み合わせは型ごとに固定で、個別に定義する
- 拡張可能レコード（匿名レコードとも）：フィールドと型の組み合わせを自由に増減させたり、フィールドの非／存在に関する多相関数を書いたりできる
- 型安全なプラグイン機構やイフェクトシステムの実装に使える
 - 前職では拡張可能レコードをベースとした型安全なプラグインシステムを実用していました

レコード

拡張可能レコード例：一部のフィールドだけに興味がある

あとからフィールドを増やす

満たすべきAPI